大数据分析实战手册

从数据探索到智能分析

竞赛培训组

2026 年 3 月 31 日

仿真平台 API 参考

接口	参数	说明
/api/data	n=1-100	实时行为数据
/api/log	lines=1-50	逐行 JSON 日志
/api/history	page, per_page, event_type, category	历史数据
/api/users	page, per_page	用户画像（1000 人）
/api/products	page, per_page, category	商品数据（500 个）
/api/stats	-	统计汇总

学习指引

前序：确保完成大数据平台部署（见部署手册）。**后续**：分析完后做可视化。**练习**：见本手册内各章节的实践练习。

目录

1 前言：什么是大数据分析？ **2**

2 1 数据准备 **3**

2.1 步骤 1：从仿真平台获取数据 3

2.2 步骤 2：将数据上传到 HDFS 4

2.3 步骤 3：在 Hive 中创建表 4

2.4 步骤 4：验证数据 5

3 2 数据探索——SQL 基础查询 **6**
3.1 基本统计函数 6
3.2 分组统计（GROUP BY） 7
3.3 条件过滤（WHERE / HAVING） 8
3.4 排序与限制 9

4 3 数据分析实战 **9**
4.1 场景 1：用户行为分析 9
4.2 场景 2：用户画像分析 10
4.3 场景 3：商品分析 11
4.4 场景 4：转化率分析（漏斗分析） 13
4.5 场景 5：时间趋势分析 14

5 4 简单机器学习（Spark MLlib） **14**
5.1 数据预处理 14
5.2 训练分类模型 15
5.3 结果解读 17

6 5 数据导出 **17**
6.1 导出分析结果 17
6.2 用 Python 做可视化 17

7 6 从不同数据源读取数据 **18**
7.1 从 HDFS 读取（通过 Hive） 18
7.2 从 HBase 读取 19
7.3 PySpark 读取 HDFS JSON 19

8 7 Spark DataFrame 操作 **20**
8.1 基本操作 20
8.2 数据转换 20
8.3 多表关联（JOIN） 21

9 8 用户画像与商品关联分析 **22**
9.1 用户画像 + 行为关联 22
9.2 商品分析 + 销量关联 23

10 9 更多分析场景 **23**
10.1 场景 6：RFM 用户价值分析 23
10.2 场景 7：留存分析 24
10.3 场景 8：品类关联分析 24

11 10 数据挖掘——综合应用 **24**
11.1 案例：用户购买预测（分类） 25
11.2 案例：用户分群（聚类） 26

12 附录：SQL 速查表 **27**

1 前言：什么是大数据分析？

用体检报告理解大数据分析

你去医院体检，医生给你做了一系列检查（血压、血糖、心率...）。这些检查结果就是”数据”。
医生做三件事：

1. **看数据**：血压多少、血糖多少（数据探索）
2. **找规律**：血压偏高的人通常体重也偏高（关联分析）
3. **下结论**：建议你少吃甜食、多运动（分析结论）

大数据分析就是同样的过程——用计算机代替医生，对海量数据进行”看数据 → 找规律 → 下结论”。

竞赛中的数据分析考什么？

根据竞赛评分标准，数据分析占 **30 分**。要求你：

- 在已部署的大数据平台上，用 SQL 查询数据
- 对数据进行多维度统计分析
- 发现数据中的规律和趋势
- 得出有价值的分析结论

你需要掌握的工具：

- **Hive SQL**：在 HDFS 上用 SQL 查询数据（适合离线大批量分析）
- **Spark SQL**：用 Spark 引擎执行 SQL（比 Hive 快，适合中等规模分析）
- **PySpark**：用 Python 调用 Spark（适合做数据处理 + 机器学习）

分析流程一图看懂

```
[HDFS数据] -> [Hive建表] -> [SQL分析] -> [导出] -> [可视化]
  原材料        标签系统      医生诊断      结果      可视化
```

2　1 数据准备

考试提醒

考试时数据由题目提供（不是仿真平台的数据），但操作流程完全一样：

1. 把题目给的数据文件上传到 HDFS
2. 在 Hive 中创建外部表
3. 用 SQL 查询分析
4. 导出结果并可视化

本手册用仿真平台数据练习，但考试时替换为题目给的数据即可。

这一章要做什么？

数据分析的前提是”数据已经存好了”。这一章教你：

1. 从云端仿真平台获取数据
2. 把数据上传到 HDFS（分布式文件系统）
3. 在 Hive 中创建表格（让数据可以用 SQL 查询）
4. 验证数据是否正确

2.1　步骤 1：从仿真平台获取数据

我们的数据来自云端电商数据仿真平台，平台每秒自动生成用户行为数据。

```
# 【在 node1 上执行】从仿真平台获取数据
# 获取 1000 条数据，保存为本地 JSON 文件
curl -s "http://163.7.1.176:8889/api/data?n=1000" > /tmp/
    user_behavior.json

# 查看前几条数据
head -c 500 /tmp/user_behavior.json
```

2.2 步骤 2：将数据上传到 HDFS

为什么要上传到 HDFS？

HDFS 是分布式文件系统，数据存到 HDFS 上后，所有节点都能访问。Hive 和 Spark 也都是从 HDFS 上读取数据。

类比：HDFS 是图书馆的书架，Hive 是图书馆的目录卡片，Spark 是图书馆的分析员。

```
# 【在 node1 上执行】创建 HDFS 目录
hdfs dfs -mkdir -p /user/data/ecommerce

# 上传数据到 HDFS
hdfs dfs -put /tmp/user_behavior.json /user/data/ecommerce/

# 验证上传成功
hdfs dfs -ls /user/data/ecommerce/
# 应该看到 user_behavior.json 文件

# 查看文件前几行
hdfs dfs -cat /user/data/ecommerce/user_behavior.json | head -5
```

2.3 步骤 3：在 Hive 中创建表

Hive 建表是什么意思？

HDFS 上的文件是”裸数据”（纯文本），不能直接用 SQL 查询。

Hive 建表就是给这些数据”贴标签”——告诉 Hive：

- 第 1 列叫 timestamp，是字符串类型
- 第 2 列叫 user_id，是字符串类型
- 第 3 列叫 event_type，是字符串类型
- ... 依此类推

建表之后，你就可以写 SQL 查询了：`SELECT * FROM user_behavior WHERE event_type='buy'`

```
-- 【在 node1 上执行】启动 Hive
```

```
hive

-- 创建数据库
CREATE DATABASE IF NOT EXISTS ecommerce;
USE ecommerce;

-- 创建外部表（数据在 HDFS 上，表只是"标签"）
CREATE EXTERNAL TABLE user_behavior (
    user_id STRING,          -- 用户ID
    event_type STRING,       -- 事件类型：view/click/add_cart/buy/search/collect/share
    session_id STRING,       -- 会话ID
    device STRING,           -- 设备：iOS/Android/PC/小程序/H5
    ip STRING,               -- IP地址
    product_id STRING,       -- 商品ID
    category STRING,         -- 商品类别
    price DOUBLE,            -- 商品价格
    page_source STRING,      -- 来源页面
    stay_time INT            -- 停留秒数
)
ROW FORMAT SERDE
    'org.apache.hive.hcatalog.data.JsonSerDe'
LOCATION '/user/data/ecommerce/';

-- 【注意】如果报错 ''JsonSerDe not found''，先执行：
-- ADD JAR /opt/hive/hcatalog/share/hcatalog/hive-hcatalog-core-3.1.3.jar;
```

常见错误

- 报错 JsonSerDe not found：需要在 Hive 中添加 JSON SerDe jar 包
- 解决办法：ADD JAR $HIVE_HOME/hcatalog/.../hive-hcatalog-core-*.jar;

2.4 步骤 4：验证数据

```
-- 查看表结构
DESCRIBE user_behavior;

```

```sql
-- 查看前10条数据
SELECT * FROM user_behavior LIMIT 10;

-- 统计总记录数
SELECT COUNT(*) FROM user_behavior;

-- 查看有哪些事件类型
SELECT DISTINCT event_type FROM user_behavior;

-- 查看有哪些商品类别
SELECT DISTINCT category FROM user_behavior;
```

动手练习

1. 从仿真平台获取 5000 条数据
2. 上传到 HDFS
3. 在 Hive 中建表并验证
4. 执行以上 5 个 SQL 查询，观察结果

3 2 数据探索——SQL 基础查询

这一章要做什么？

学会用 SQL 对数据进行基本的统计分析。这是数据分析的基本功，就像医生要学会看体检报告一样。

3.1 基本统计函数

```sql
-- 统计总记录数
SELECT COUNT(*) AS 总记录数 FROM user_behavior;

-- 统计去重后的用户数
SELECT COUNT(DISTINCT user_id) AS 独立用户数 FROM user_behavior;

-- 统计去重后的商品数
SELECT COUNT(DISTINCT product_id) AS 独立商品数 FROM
    user_behavior;
```

```sql

-- 价格统计（平均值、最大值、最小值）
SELECT
    AVG(price) AS 平均价格,
    MAX(price) AS 最高价格,
    MIN(price) AS 最低价格,
    STDDEV(price) AS 价格标准差
FROM user_behavior
WHERE price IS NOT NULL;

-- 停留时间统计
SELECT
    AVG(stay_time) AS 平均停留秒数,
    MAX(stay_time) AS 最长停留秒数,
    MIN(stay_time) AS 最短停留秒数
FROM user_behavior
WHERE stay_time IS NOT NULL;
```

3.2 分组统计（GROUP BY）

GROUP BY 是什么？

GROUP BY = "按某个维度分组，然后对每组做统计"。

类比：老师把全班同学按"性别"分组，然后统计每组的平均成绩。这就是 GROUP BY。

```sql
-- 按事件类型分组，统计每种事件的数量
SELECT
    event_type AS 事件类型,
    COUNT(*) AS 数量
FROM user_behavior
GROUP BY event_type
ORDER BY 数量 DESC;

-- 按商品类别分组，统计每类的平均价格
SELECT
    category AS 品类,
    COUNT(*) AS 记录数,
    ROUND(AVG(price), 2) AS 平均价格
```

```sql
FROM user_behavior
WHERE price IS NOT NULL
GROUP BY category
ORDER BY 记录数 DESC;

-- 按设备类型分组
SELECT
    device AS 设备,
    COUNT(*) AS 使用次数,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM user_behavior
        ), 2) AS 占比百分比
FROM user_behavior
GROUP BY device
ORDER BY 使用次数 DESC;
```

3.3 条件过滤（WHERE / HAVING）

```sql
-- WHERE: 过滤行（分组前过滤）
-- 查看所有"购买"事件
SELECT * FROM user_behavior WHERE event_type = 'buy';

-- 查看价格大于 5000 的记录
SELECT * FROM user_behavior WHERE price > 5000;

-- 查看 iOS 用户的行为
SELECT * FROM user_behavior WHERE device = 'iOS';

-- 多条件过滤（AND / OR）
SELECT * FROM user_behavior
WHERE event_type = 'buy' AND price > 1000;

-- HAVING: 过滤组（分组后过滤）
-- 只显示记录数大于 100 的品类
SELECT
    category,
    COUNT(*) AS 数量
FROM user_behavior
GROUP BY category
```

```sql
HAVING COUNT(*) > 100
ORDER BY 数量 DESC;
```

3.4 排序与限制

```sql
-- ORDER BY: 排序
-- 按价格降序排列 (从高到低)
SELECT * FROM user_behavior
WHERE price IS NOT NULL
ORDER BY price DESC
LIMIT 20;

-- 按停留时间升序排列 (从低到高)
SELECT * FROM user_behavior
WHERE stay_time IS NOT NULL
ORDER BY stay_time ASC
LIMIT 20;
```

4 3 数据分析实战

这一章要做什么？

用真实的电商数据，做 5 个具体的分析场景。每个场景都包含：业务问题 → SQL 查询 → 结果解读。
这是竞赛中最可能考到的内容！

4.1 场景 1：用户行为分析

业务问题

老板问："我们的用户在网站上都在做什么？哪些行为最多？"

```sql
-- 分析1: 各种行为的占比
SELECT
    event_type AS 行为类型,
    COUNT(*) AS 次数,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM user_behavior
        ), 2) AS 占比
```

```sql
FROM user_behavior
GROUP BY event_type
ORDER BY 次数 DESC;

-- 分析2: 每种行为的平均价格
SELECT
    event_type AS 行为类型,
    ROUND(AVG(price), 2) AS 平均价格,
    ROUND(MIN(price), 2) AS 最低价格,
    ROUND(MAX(price), 2) AS 最高价格
FROM user_behavior
WHERE price IS NOT NULL
GROUP BY event_type
ORDER BY 平均价格 DESC;

-- 分析3: 各品类的行为分布
SELECT
    category AS 品类,
    event_type AS 行为,
    COUNT(*) AS 次数
FROM user_behavior
GROUP BY category, event_type
ORDER BY category, 次数 DESC;
```

4.2 场景 2: 用户画像分析

业务问题

老板问: "我们的用户都是什么人? 用什么设备? 在哪些城市? "

```sql
-- 分析1: 设备分布
SELECT
    device AS 设备类型,
    COUNT(*) AS 使用次数,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM user_behavior
        ), 2) AS 占比
FROM user_behavior
GROUP BY device
ORDER BY 使用次数 DESC;
```

```sql

-- 分析2: 来源页面分布
SELECT
    page_source AS 来源页面,
    COUNT(*) AS 访问次数,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM user_behavior
        ), 2) AS 占比
FROM user_behavior
GROUP BY page_source
ORDER BY 访问次数 DESC;

-- 分析3: 品类偏好 (用户最关注哪些品类)
SELECT
    category AS 品类,
    COUNT(DISTINCT user_id) AS 独立用户数,
    COUNT(*) AS 总行为数
FROM user_behavior
GROUP BY category
ORDER BY 独立用户数 DESC;
```

4.3 场景 3：商品分析

业务问题

老板问："哪些商品最受欢迎？哪个品类卖得最好？"

```sql
-- 分析1: 热销商品 TOP 10
SELECT
    product_id AS 商品ID,
    category AS 品类,
    COUNT(*) AS 被操作次数,
    SUM(CASE WHEN event_type = 'buy' THEN 1 ELSE 0 END) AS 购买
        次数,
    SUM(CASE WHEN event_type = 'view' THEN 1 ELSE 0 END) AS 浏览
        次数
FROM user_behavior
GROUP BY product_id, category
ORDER BY 购买次数 DESC, 被操作次数 DESC
LIMIT 10;
```

```sql

-- 分析2: 品类销量排行
SELECT
    category AS 品类,
    SUM(CASE WHEN event_type = 'buy' THEN 1 ELSE 0 END) AS 购买次数,
    SUM(CASE WHEN event_type = 'view' THEN 1 ELSE 0 END) AS 浏览次数,
    ROUND(SUM(CASE WHEN event_type = 'buy' THEN 1 ELSE 0 END) * 100.0 /
        SUM(CASE WHEN event_type = 'view' THEN 1 ELSE 0 END), 2) AS 转化率
FROM user_behavior
GROUP BY category
ORDER BY 购买次数 DESC;

-- 分析3: 价格区间分布
SELECT
    CASE
        WHEN price < 100 THEN '0-100元'
        WHEN price < 500 THEN '100-500元'
        WHEN price < 1000 THEN '500-1000元'
        WHEN price < 5000 THEN '1000-5000元'
        ELSE '5000元以上'
    END AS 价格区间,
    COUNT(*) AS 商品数量,
    ROUND(AVG(price), 2) AS 平均价格
FROM user_behavior
WHERE price IS NOT NULL
GROUP BY
    CASE
        WHEN price < 100 THEN '0-100元'
        WHEN price < 500 THEN '100-500元'
        WHEN price < 1000 THEN '500-1000元'
        WHEN price < 5000 THEN '1000-5000元'
        ELSE '5000元以上'
    END
ORDER BY 平均价格;
```

4.4 场景 4：转化率分析（漏斗分析）

业务问题

老板问："100 个浏览的人，有多少人加了购物车？有多少人最终买了？"
这就是"漏斗分析"——从浏览 → 加购 → 购买，每一步有多少人流失。

```sql
-- 漏斗分析：浏览 -> 加购 -> 购买
SELECT
    '1.浏览' AS 步骤,
    COUNT(DISTINCT user_id) AS 用户数
FROM user_behavior WHERE event_type = 'view'
UNION ALL
SELECT
    '2.加购' AS 步骤,
    COUNT(DISTINCT user_id) AS 用户数
FROM user_behavior WHERE event_type = 'add_cart'
UNION ALL
SELECT
    '3.购买' AS 步骤,
    COUNT(DISTINCT user_id) AS 用户数
FROM user_behavior WHERE event_type = 'buy';

-- 各品类转化率
SELECT
    category AS 品类,
    COUNT(DISTINCT CASE WHEN event_type='view' THEN user_id END) AS 浏览用户,
    COUNT(DISTINCT CASE WHEN event_type='add_cart' THEN user_id END) AS 加购用户,
    COUNT(DISTINCT CASE WHEN event_type='buy' THEN user_id END) AS 购买用户,
    ROUND(COUNT(DISTINCT CASE WHEN event_type='buy' THEN user_id END) * 100.0 /
          NULLIF(COUNT(DISTINCT CASE WHEN event_type='view' THEN user_id END), 0), 2)
          AS 浏览到购买转化率
FROM user_behavior
GROUP BY category
ORDER BY 浏览到购买转化率 DESC;
```

4.5 场景 5：时间趋势分析

业务问题

老板问：”用户一般什么时候最活跃？上午还是下午？”

```sql
-- 按小时统计活跃度（需要先从 timestamp 提取小时）
SELECT
    SUBSTR(timestamp, 12, 2) AS 小时,
    COUNT(*) AS 行为次数,
    COUNT(DISTINCT user_id) AS 独立用户数
FROM user_behavior
GROUP BY SUBSTR(timestamp, 12, 2)
ORDER BY 小时;

-- 按日期统计趋势
SELECT
    SUBSTR(timestamp, 1, 10) AS 日期,
    COUNT(*) AS 行为次数,
    COUNT(DISTINCT user_id) AS 独立用户数,
    SUM(CASE WHEN event_type = 'buy' THEN 1 ELSE 0 END) AS 购买
        次数
FROM user_behavior
GROUP BY SUBSTR(timestamp, 1, 10)
ORDER BY 日期;
```

5 4 简单机器学习（Spark MLlib）

这一章要做什么？

用 Spark 的机器学习库（MLlib），对用户数据做一个简单的预测模型。比如”预测用户是否会购买”。

类比：医生根据体检数据（特征），预测你是否会得某种病（标签）。这里我们根据用户行为数据（特征），预测用户是否会购买（标签）。

5.1 数据预处理

```bash
# 【在 node1 上执行】启动 pyspark
pyspark
```

```python

from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# 读取 HDFS 上的数据
spark = SparkSession.builder.appName("UserAnalysis").getOrCreate()
df = spark.read.json("hdfs://node1:9000/user/data/ecommerce/user_behavior.json")

# 查看数据
df.show(5)
df.printSchema()

# 创建特征：每个用户的行为统计
user_features = df.groupBy("user_id").agg(
    count("*").alias("总行为数"),
    sum(when(col("event_type") == "view", 1).otherwise(0)).alias("浏览次数"),
    sum(when(col("event_type") == "click", 1).otherwise(0)).alias("点击次数"),
    sum(when(col("event_type") == "add_cart", 1).otherwise(0)).alias("加购次数"),
    sum(when(col("event_type") == "buy", 1).otherwise(0)).alias("购买次数"),
    avg("price").alias("平均价格"),
    avg("stay_time").alias("平均停留时间")
)

# 创建标签：是否购买过（1=是，0=否）
user_features = user_features.withColumn(
    "是否购买", when(col("购买次数") > 0, 1).otherwise(0)
)

user_features.show(10)
```

5.2 训练分类模型

```python
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# 1. 组装特征向量（MLlib 要求特征放在一个向量列中）
assembler = VectorAssembler(
    inputCols=["总行为数", "浏览次数", "点击次数", "加购次数", "平均价格", "平均停留时间"],
    outputCol="features"
)
data = assembler.transform(user_features)

# 2. 拆分训练集和测试集（70%训练，30%测试）
train, test = data.randomSplit([0.7, 0.3], seed=42)
print(f"训练集：{train.count()} 条，测试集：{test.count()} 条")

# 3. 训练模型（逻辑回归）
lr = LogisticRegression(featuresCol="features", labelCol="是否购买")
model = lr.fit(train)

# 4. 在测试集上预测
predictions = model.transform(test)

# 5. 评估模型准确率
evaluator = BinaryClassificationEvaluator(labelCol="是否购买")
auc = evaluator.evaluate(predictions)
print(f"模型 AUC: {auc:.4f}")

# 查看预测结果
predictions.select("user_id", "是否购买", "prediction", "probability").show(10)
```

5.3 结果解读

AUC 是什么意思？

AUC（Area Under Curve）是分类模型的评价指标，取值 0-1：

- AUC = 0.5：模型和瞎猜一样（没用）
- AUC = 0.7：还可以
- AUC = 0.8：不错
- AUC = 0.9+：很好

6 5 数据导出

6.1 导出分析结果

```
# 【在 node1 上执行】将 Hive 查询结果导出为 CSV
hive -e "
SET hive.resultset.use.unique.column.names=false;
SELECT event_type, COUNT(*) as cnt
FROM ecommerce.user_behavior
GROUP BY event_type
ORDER BY cnt DESC
" | sed 's/\t/,/g' > /tmp/event_stats.csv

# 上传到 HDFS 或下载到本地
cat /tmp/event_stats.csv
```

6.2 用 Python 做可视化

```
# 【在本地电脑上执行】用 matplotlib 画图
import matplotlib.pyplot as plt
import pandas as pd

# 读取分析结果
df = pd.read_csv('/tmp/event_stats.csv')

# 柱状图
```

```python
plt.figure(figsize=(10, 6))
plt.bar(df['event_type'], df['cnt'], color='steelblue')
plt.title('用户行为分布')
plt.xlabel('行为类型')
plt.ylabel('次数')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('event_distribution.png', dpi=150)
plt.show()
```

7 6 从不同数据源读取数据

为什么要学多种数据源？

大数据平台上的数据可能存储在不同地方：

- **HDFS**：原始文件（CSV、JSON、日志）
- **HBase**：结构化的用户/商品信息
- **ClickHouse**：已经统计好的聚合数据

你需要知道”从哪里读、怎么读”。

7.1 从 HDFS 读取（通过 Hive）

```sql
-- Hive 通过 LOCATION 指向 HDFS 上的文件
-- 建表时指定 HDFS 路径，之后就可以用 SQL 查询
CREATE EXTERNAL TABLE raw_logs (
    line STRING
) LOCATION '/user/data/logs/';

-- 也可以用正则提取字段
CREATE EXTERNAL TABLE parsed_logs (
    user_id STRING, action STRING, time_str STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'
WITH SERDEPROPERTIES (
    "input.regex" = "(\\S+)\\s+(\\S+)\\s+(\\S+)"
)
```

```
LOCATION '/user/data/logs/';
```

7.2 从 HBase 读取

```sql
-- 在 Hive 中创建外部表映射 HBase
CREATE EXTERNAL TABLE user_profile_hbase (
    rowkey STRING,
    name STRING,
    age INT,
    city STRING
)
STORED BY 'org.apache.hadoop.hive.hbase.HBaseStorageHandler'
WITH SERDEPROPERTIES (
    "hbase.columns.mapping" = ":key,info:name,info:age,info:city"
)
TBLPROPERTIES ("hbase.table.name" = "user_profile");

-- 现在可以用 SQL 查询 HBase 数据了
SELECT city, AVG(age) FROM user_profile_hbase GROUP BY city;
```

7.3 PySpark 读取 HDFS JSON

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Analysis").getOrCreate()

# 读取 HDFS 上的 JSON 文件
df = spark.read.json("hdfs://node1:9000/user/data/ecommerce/user_behavior.json")

# 查看数据
df.show(5)            # 前5行
df.printSchema()      # 列名和类型
df.describe().show()  # 统计摘要
df.count()            # 总行数
```

8 7 Spark DataFrame 操作

Spark DataFrame 是什么？

Spark DataFrame = " 分布式版本的 Pandas DataFrame"。语法非常相似，但数据分布在多台机器上并行处理。

类比：Pandas 是一个工人干活，Spark 是一群工人同时干活。

8.1 基本操作

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

spark = SparkSession.builder.getOrCreate()
df = spark.read.json("hdfs://node1:9000/user/data/ecommerce/user_behavior.json")

# 选择列
df.select("user_id", "event_type", "price").show(5)

# 过滤行
df.filter(col("event_type") == "buy").show(5)
df.filter(col("price") > 1000).show(5)
df.filter((col("event_type") == "buy") & (col("price") > 1000)).show(5)

# 分组统计
df.groupBy("event_type").count().show()
df.groupBy("category").agg(
    avg("price").alias("平均价格"),
    count("*").alias("记录数")
).show()

# 排序
df.orderBy(col("price").desc()).show(10)
```

8.2 数据转换

```python
# 新增列
df = df.withColumn("price_level",
    when(col("price") < 100, "低价")
    .when(col("price") < 1000, "中价")
    .otherwise("高价")
)

# 删除列
df = df.drop("ip")

# 重命名列
df = df.withColumnRenamed("event_type", "事件类型")

# 类型转换
df = df.withColumn("price", col("price").cast("double"))
```

8.3 多表关联（JOIN）

```python
# 假设有两个表：用户行为表 和 商品信息表
behavior = spark.read.json("hdfs://node1:9000/user/data/ecommerce/user_behavior.json")
# products 需要先从 HDFS 读取（或从 Hive 查询）

# 内连接：只保留两个表都匹配的行
joined = behavior.join(products, on="product_id", how="inner")

# 左连接：保留左表所有行
joined = behavior.join(products, on="product_id", how="left")

# 关联后统计
joined.groupBy("category").agg(
    count("*").alias("总行为数"),
    sum(when(col("event_type")=="buy", 1).otherwise(0)).alias("购买数")
).show()
```

9 8 用户画像与商品关联分析

这一章做什么？

除了分析行为数据，还要结合「用户表」和「商品表」做跨表分析。仿真平台提供三张表：

- **行为表**：326,000+ 条用户行为记录
- **用户表**：1000 个用户画像（年龄/城市/会员等级）
- **商品表**：500 个商品（品类/价格/评分/销量）

9.1 用户画像 + 行为关联

```python
# 获取三种数据
df_behavior = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/data?n=5000").json())
df_users = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/users?per_page=1000").json()['data'])
df_products = pd.DataFrame(requests.get("http://163.7.1.176:8889/api/products?per_page=500").json()['data'])

# 按 user_id 关联
merged1 = pd.merge(df_behavior, df_users, on='user_id', how='left')

# 不同城市的用户购买偏好
city_buy = merged1.groupby('city')['event_type'].value_counts().unstack(fill_value=0)
print(city_buy.sort_values(by='buy', ascending=False).head())

# 不同会员等级消费
level_stats = merged1[merged1['event_type']=='buy'].groupby('member_level').agg({
    'price': ['count', 'mean', 'sum']
})
print(level_stats)
```

9.2 商品分析 + 销量关联

```python
# 按 product_id 关联
merged2 = pd.merge(df_behavior, df_products, on='product_id',
    how='left')

# 不同品类被购买次数 TOP10
top = merged2[merged2['event_type']=='buy'].groupby(
    ['category_y', 'name_y']).size().reset_index(name='购买次数'
        )
print(top.sort_values('购买次数', ascending=False).head(10))
```

10 9 更多分析场景

10.1 场景 6：RFM 用户价值分析

什么是 RFM？

RFM 是衡量用户价值的经典模型：

- **R**（Recency）：最近一次消费距今多久（越近越好）
- **F**（Frequency）：消费频率（越多越好）
- **M**（Monetary）：消费金额（越高越好）

```sql
-- 在 Hive 中计算 RFM
SELECT
    user_id,
    MAX(timestamp) AS 最近消费时间,
    COUNT(CASE WHEN event_type='buy' THEN 1 END) AS 消费次数,
    SUM(CASE WHEN event_type='buy' THEN price ELSE 0 END) AS 消
        费总额
FROM user_behavior
GROUP BY user_id
ORDER BY 消费总额 DESC
LIMIT 20;
```

10.2 场景 7：留存分析

什么是留存？

留存 = ”昨天来过的用户，今天还来吗？”。留存率越高，说明用户粘性越好。

```sql
-- 按日期统计独立用户数和新用户数
SELECT
    SUBSTR(timestamp, 1, 10) AS 日期,
    COUNT(DISTINCT user_id) AS 独立用户数
FROM user_behavior
GROUP BY SUBSTR(timestamp, 1, 10)
ORDER BY 日期;
```

10.3 场景 8：品类关联分析

```sql
-- 找出被同一批用户关注最多的品类对
SELECT
    a.category AS 品类A,
    b.category AS 品类B,
    COUNT(DISTINCT a.user_id) AS 共同用户数
FROM user_behavior a
JOIN user_behavior b
    ON a.user_id = b.user_id
    AND a.category < b.category
WHERE a.event_type = 'buy' AND b.event_type = 'buy'
GROUP BY a.category, b.category
ORDER BY 共同用户数 DESC
LIMIT 20;
```

11 10 数据挖掘——综合应用

综合应用考什么？

题目会给你一个数据集和一个问题，要求你综合运用预处理 + 分析 + 挖掘来求解。

典型题目：

- ”分析用户行为数据，预测哪些用户会流失”
- ”分析商品销售数据，找出关联商品”
- ”分析用户画像数据，将用户分群”

解题思路：

1. 读取数据（HDFS → Hive → Spark）
2. 数据预处理（清洗 + 特征工程）
3. 选择算法（分类/聚类/关联）
4. 训练模型（Spark MLlib）
5. 评估结果 + 可视化

数据挖掘是什么意思？

数据挖掘 = 从大量数据中发现隐藏的规律和模式。竞赛要求”综合应用数据预处理、分析与挖掘技术求解给定问题”。

常见考点：

- 分类：预测用户是否会购买（是/否）
- 聚类：将用户分为若干群体
- 关联规则：买了 A 的人通常也会买 B
- 回归：预测某个连续值（如商品价格）

11.1 案例：用户购买预测（分类）

```python
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import VectorAssembler

# 1. 构建特征：每个用户的统计信息
user_features = df.groupBy("user_id").agg(
    count("*").alias("总行为数"),
    sum(when(col("event_type") == "buy", 1).otherwise(0)).alias("购买次数"),
    avg("price").alias("平均价格")
```

```
)

# 2. 创建标签：是否购买过
user_features = user_features.withColumn("label",
    when(col("购买次数") > 0, 1).otherwise(0))

# 3. 组装特征向量
assembler = VectorAssembler(
    inputCols=["总行为数", "购买次数", "平均价格"],
    outputCol="features")
data = assembler.transform(user_features)

# 4. 拆分训练/测试集
train, test = data.randomSplit([0.7, 0.3], seed=42)

# 5. 训练模型
lr = LogisticRegression(featuresCol="features", labelCol="label"
    )
model = lr.fit(train)

# 6. 预测和评估
predictions = model.transform(test)
predictions.select("user_id", "label", "prediction").show(10)

# 7. 计算准确率
correct = predictions.filter(col("label") == col("prediction")).
    count()
total = predictions.count()
print(f"准确率：{correct/total:.2%}")
```

11.2 案例：用户分群（聚类）

```
from pyspark.ml.clustering import KMeans

# 构建用户特征
user_features = df.groupBy("user_id").agg(
    count("*").alias("总行为"),
    sum(when(col("event_type") == "view", 1).otherwise(0)).alias
```

```
        ("浏览"),
    sum(when(col("event_type") == "buy", 1).otherwise(0)).alias("购买"),
    avg("price").alias("平均价格")
)

# 组装特征
assembler = VectorAssembler(
    inputCols=["总行为", "浏览", "购买", "平均价格"],
    outputCol="features")
data = assembler.transform(user_features)

# K-Means 聚类（分 3 群）
kmeans = KMeans(featuresCol="features", k=3, seed=42)
model = kmeans.fit(data)

# 查看聚类结果
result = model.transform(data)
result.groupBy("prediction").agg(
    count("*").alias("用户数"),
    avg("总行为").alias("平均行为数"),
    avg("购买").alias("平均购买数")
).show()
```

12 附录：SQL 速查表

功能	SQL	说明
查询全部	SELECT * FROM t	查看所有数据
条件查询	SELECT * FROM t WHERE x=1	过滤行
分组计数	SELECT x, COUNT(*) FROM t GROUP BY x	按 x 分组计数
排序	SELECT * FROM t ORDER BY x DESC	降序排列
限制条数	SELECT * FROM t LIMIT 10	只取前 10 条
去重	SELECT DISTINCT x FROM t	去除重复值
聚合函数	COUNT/SUM/AVG/MAX/MIN	统计函数
条件聚合	SUM(CASE WHEN x THEN 1 ELSE 0 END)	条件计数
连接	SELECT * FROM t1 JOIN t2 ON t1.id=t2.id	多表连接
子查询	SELECT * FROM (SELECT ...)	嵌套查询